김민혁 | FRONTEND DEVELOPER

성격은 둥글게,
맡은 일은 끝까지 성실하게

동료에게는 편하게 말 걸 수 있는 사람으로, 문제는 막힐수록 더 찾아보고 부딪쳐 봅니다.
가설이 빗나가도 조급해하지 않고, 더 나은 구조가 보이면 내가 만든 코드도 고집하지 않습니다.

CAREER
B2B2C 서비스 프론트엔드
2024.05 – 2025.09 · 1년 5개월 · 단독 개발

SIDE PROJECT
YAMOYO · SWYP 웹 12기 대상
커밋 381 / 전체 591 · 머지 PR 39 / 전체 71

STACK
Next.js · React · TypeScript
TanStack Query · Zustand · Tiptap · WebSocket(STOMP)

EMAIL
jaqwe2301@gmail.com

GITHUB
github.com/jaqwe2301

BLOG
mini-frontend.tistory.com

CAREER B2B2C 서비스 프론트엔드 개발 · 2024.05 – 2025.09 · 프론트엔드 단독 개발

01 COMPATIBILITY
인증 콜백 처리
일부 환경에서만 나타난 500 에러, Next.js Route Handler로 GET · POST 인증 콜백을 모두 처리
요청 흐름 추적 · 인앱 브라우저 호환 · 요청 방식별 분기
PAGE 04–05

02 EDITOR
에디터 확장 구조
DOM 기반 자체 에디터의 한계를 확인하고, 핵심 편집 로직을 Tiptap 커스텀 Extension으로 이식
DOM 구조 분석 · Tiptap · ProseMirror · Custom Extension
PAGE 06–08

03 PERFORMANCE
학과 초성 검색
초성 검색이 안 되는 공공 API, 학과 9,329개를 정적 데이터로 다루기
빌드 타임 정적 데이터 생성 · 지연 렌더링(Lazy Rendering) · 네트워크 의존 최소화
PAGE 09

SIDE PROJECT YAMOYO · 2025.12 – 2026.05 · FE 2 · BE 3 · DE 3 · PM 1 · SWYP 웹 12기 대상

04 YAMOYO
실시간 연결 · 인증 흐름
WebSocket 실시간 동기화와 연결 유지, 이벤트 기반 인증 라우팅 설계
STOMP · SockJS · Event Bus · 중복 재발급 방지
PAGE 11–15

2024.05 – 2025.09

CAREER

B2B2C 서비스 프론트엔드 개발

1년 5개월 · 프론트엔드 단독 개발 · Next.js App Router · TypeScript

IN THIS PART

01 COMPATIBILITY
일부 환경에서만 나타난 본인인증 500 에러 P.04–05

02 EDITOR
DOM 기반 자체 에디터에서 Tiptap 확장으로 P.06–08

03 PERFORMANCE
초성 검색이 안 되는 공공 API,
학과 9,329개를 정적 데이터로 다루기 P.09

본인인증은 끝났는데 돌아온 건 500 에러, 왜 일부 환경에서만 나타났을까?

NICE 휴대폰 본인인증 연동 과정에서, iOS Safari와 카카오톡 인앱 브라우저에서만 인증 마지막 단계가 실패했습니다.
새 창을 띄우는 방식을 의심했지만 해결되지 않았고, 실제로 오가는 요청을 확인해 원인을 찾았습니다.

반려동물 케어 커뮤니티 | 프론트엔드 단독 개발 | Next.js App Router · TypeScript | NICE 본인인증

A 증상 NICE 휴대폰 본인인증을 완료한 뒤, 일부 환경에서만 500 에러가 돌아오는 문제 — BEFORE

STEP 1
NICE 인증 접속
가입 화면에서 본인인증을 시작

구현 인증에 필요한 값을 채운 폼을 만들고, NICE 인증 화면을 별도 창으로 열어 전송

→

STEP 2
휴대폰 번호 입력
통신사 · 이름 · 번호를 입력

→

STEP 3
인증번호 입력 후 확인
문자로 받은 번호를 입력하고 확인

구현 인증 절차는 NICE 화면에서 진행되고, 끝나면 그 결과가 서비스 주소로 되돌아옴

→

여기서 환경에 따라 결과가 갈림
인증 자체는 끝났는데, 돌아오는 길에서만 실패

✓ **인증 완료**
데스크탑 브라우저 · 안드로이드 Chrome

✕ **확인 버튼에서 500 에러**
iOS Safari · 카카오톡 인앱 브라우저

인증 자체는 끝나는데 **돌아오는 요청에서만 실패**했고, 재현되는 환경이 한정돼 개발 중이던 데스크탑에서는 문제가 드러나지 않았습니다.

B 원인 추적 브라우저를 판별해 우회하는 방법을 먼저 시도했고, 실패를 확인한 뒤 추측을 멈추고 실제 요청을 직접 관측했습니다 — PROCESS

가설 새 창이 문제일 것
모바일에서는 **팝업 차단과 창 전환**이 까다로우니, 새 창을 띄우는 방식 자체가 원인이라고 봤습니다.

팝업을 없애고 같은 창에서 처리하면
해결될 것으로 예상했습니다.

→

검증 같은 창에서 열어보기
User-Agent로 브라우저를 판별해, 모바일에서는 팝업 대신 현재 창으로 전송하도록 바꿨습니다.

```
const isMobile = /Mobi|iPhone|KAKAO/i
  .test(navigator.userAgent);

if (isMobile) form.target = "_self";
```

✕ 여전히 같은 오류 — 가설 기각

→

관측 오가는 요청을 직접 확인
개발자 도구 네트워크 탭에서 **인증 완료 직후 들어오는 요청**의 메서드와 페이로드를 열어봤습니다.

! 실패한 환경에서는 결과가 주소가 아니라
POST 요청 본문(form data)으로 들어온다

ROOT CAUSE 원인 도출

원인은 환경이 아니라, 인증 결과가 돌아오는 HTTP 요청 방식(GET / POST)이 달랐던 것

도출 개발자 도구 네트워크 탭에서 인증 완료 직후 요청의 **메서드와 페이로드** 직접 확인 — 성공한 환경은 GET 쿼리스트링, 실패한 환경은 POST form data로 결과 도착

결론 결과 전송 방식(GET/POST)의 **결정권은 인증 기관에 있어 프론트엔드는 선택 불가** — 화면을 그리는 페이지 컴포넌트는 GET 렌더링만 처리해 POST에서 500 발생 → **요청 방식별 분기가 가능한 Route Handler**로의 이관 필요

GET일지 POST일지 모르는 인증 결과, Next.js의 Route Handler에서 모두 받아내기

인증 결과가 어떤 방식으로 돌아올지는 인증 기관이 정하고, 화면을 그리는 페이지는 POST 본문을 읽지 못했습니다.
결과를 받는 위치를 요청 방식별로 분기할 수 있는 서버 엔드포인트로 옮겨, 두 방식 모두 같은 인증 결과로 처리했습니다.

A 수신 위치 선택

결과를 보여줄 화면이 이미 있는데, 왜 그 화면에서 받지 않았는가 — WHY

Q 결과 화면인 `page.tsx`가 이미 있는데, 왜 그 페이지에서 인증 결과를 받지 않았나?

A

01 결과가 돌아오는 **URL 경로는 하나**, 그 경로로 **GET · POST가 모두** 도착

02 `page.tsx`는 **GET 렌더링**만 처리 — POST 본문은 읽을 수 없음

03 `route.ts`는 한 경로에서 **GET · POST를 각각 export** 가능

→ 한 경로에서 두 방식을 모두 받으려면 route.ts(Route Handler)

	`page.tsx` 페이지 컴포넌트	`route.ts` Route Handler
역할	화면을 렌더링	요청을 받아 응답을 직접 생성
GET · 쿼리스트링	✓ URL 쿼리 파라미터로 읽기 가능	✓ URL 쿼리 파라미터로 읽기 가능
POST · form data	✗ 본문을 읽지 못해 500 에러	✓ 요청 본문에서 읽기 가능
요청 방식별 분기	✗ 불가	✓ `export GET` · `export POST`

B 해결 구조

Next.js의 Route Handler로 결과를 받고, 요청 방식별로 암호화된 인증 결과를 추출 — SOLUTION



TS route.ts · 하나의 경로, 두 개의 메서드

```ts
export async function POST(req: NextRequest) {
  // 페이지에서는 읽을 수 없는 본문을 여기서 꺼낸다
  const form = await req.formData();
  const enc_data = form.get("enc_data");
  return new NextResponse(html, { headers });
}

export async function GET(req: NextRequest) {
  // 쿼리스트링에 담겨 온 값을 그대로 꺼낸다
  const { searchParams } = new URL(req.url);
  const enc_data = searchParams.get("enc_data");
  return new NextResponse(html, { headers });
}
```

RESULT 개선 결과

User-Agent 판별과 폴링 없이, 하나의 Route Handler로 모든 환경의 인증 결과를 한 곳에서 처리

구조 환경별로 갈라지던 **예외 처리 분기와 폴링(주기적 상태 확인) 로직 제거** — 브라우저 판별 없이 단일 경로로 결과 수신

확장 새 전달 방식이 추가돼도 **Route Handler의 메서드 핸들러 확장**만으로 대응 — 페이지 컴포넌트 변경 없이 유지보수 범위 최소화

02 EDITOR 본문 에디터

1 직접 구현 — 2 한계와 전환 — 3 확장 이식

N사 블로그 에디터는 어떻게 동작할까? 구조 분석에서 시작한 에디터 직접 구현

커뮤니티 글쓰기에 블로그 수준의 서식 · 이미지 편집이 필요했습니다.
N사 블로그의 HTML 구조를 분석해 원리를 파악하고, 텍스트 노드 분할과 비율 유지 이미지 리사이징을 직접 구현했습니다.

반려동물 케어 커뮤니티 | 프론트엔드 단독 개발 | Next.js App Router · TypeScript | contentEditable · DOM API

A 구조 분석 N사 블로그 에디터의 HTML을 직접 열어보며 편집 동작을 역추적 ANALYSIS

글쓰기 화면

B I U T+

안녕**하세**요

이미지를 선택하면
모서리에 크기 조절 핸들

→

개발자 도구 · Elements 단순화

```
<p>
  <span>안녕</span>
  <span class="bold">하세</span>   <!-- 서식 구간만 분리 -->
  <span>요</span>
</p>
<div data-id="img-01">   <!-- 블록 단위 식별 -->
  <img style="width:320px">
</div>
```

→

01 서식은 텍스트 노드를 쪼개 적용
선택한 구간만 새 span으로 감싸고,
앞뒤 구간은 기존 서식을 그대로 유지

02 이미지는 블록 단위로 식별 · 조작
식별자로 대상 블록을 찾고,
선택된 이미지에만 조작 UI를 덧붙임

B 직접 구현 파악한 원리를 그대로 옮겨, 라이브러리 없이 DOM을 직접 다뤄 구현 BUILD

01 TEXT STYLE 텍스트 노드 분할

1 **선택 영역의 텍스트 노드 수집**
 안 녕 **하 세** 요 ← 드래그로 '하세' 선택

2 **앞 · 선택 · 뒤 구간 분리**
앞: 안 녕 / 선택: **하 세** / 뒤: 요
선택 경계에서 노드를 세 조각으로

3 **새 span 노드 생성 및 기존 노드 교체**
안녕 **하세** 요

RESULT **선택 구간만 스타일 적용** · 주변 구간은 기존 서식 유지

02 IMAGE RESIZE 비율을 유지한 이미지 크기 조절

data-id="img-01"
320 × 200 → **480 × 300**
비율 16 : 10 유지

1 **이미지 블록을 data-id로 식별**
어떤 블록을 조작할지 먼저 특정

2 **선택된 이미지에 리사이즈 핸들 생성**
네 모서리에 드래그 가능한 핸들 부착

3 **드래그한 만큼 너비 변경**
시작 좌표 대비 이동 거리로 계산

4 **높이는 원본 비율로 계산**
$height = width / (w_0 / h_0)$

RESULT **어느 모서리를 끌어도 원본 비율 유지** · 이미지 왜곡 없이 크기 조절

BUT 편집 로직을 직접 구현하며 에디터의 동작 원리를 익혔지만, **기능이 늘어날수록 예외 처리와 직접 DOM을 제어하는 범위도 함께 증가**

한계와 전환 →

ROLE 프론트엔드 단독 개발 BLOG mini-frontend.tistory.com/13 · /14

기능을 더할수록 늘어나는 예외 처리,
기본 편집은 Tiptap에, 핵심 로직은 확장으로

줄바꿈 · 붙여넣기 같은 기본 입력마다 예외 처리가 늘었고, React 상태와 실제 DOM이 서로 어긋났습니다.
문서 모델과 트랜잭션으로 동작하는 Tiptap을 도입해, 예외 처리 대신 기능 구현에 집중할 수 있는 기반을 마련했습니다.

A 직접 구현의 한계

기능을 하나 더할 때마다, 기본 편집 동작과 DOM 상태까지 함께 책임져야 하는 구조 — LIMIT

01 EDITING EXCEPTIONS 기능마다 이어지는 예외 처리

- `Enter` **줄바꿈** — 새 문단 생성 시 서식 처리
- `Ctrl V` **붙여넣기** — 외부 HTML 유입 처리
- **!** **텍스트를 입력하고 편집하는 모든 순간** — 동작마다 직접 DOM을 고쳐야 함
- `Back` **문장 시작점의 Backspace** — 앞 문단과 합쳐지는 경계 처리
- `B` **스타일 적용** — 선택 구간마다 span 분할

기본 입력 동작마다 **개별 로직 작성과 예외 처리가 반복**

02 DOM SYNCHRONIZATION React 상태와 실제 DOM의 불일치

React가 기억하는 화면

`state` `props` `Virtual DOM`

```
<p>
  안녕하세요
</p>
```

≠ 불일치

브라우저에서 직접 바뀐 화면

`span 교체` `스타일 수정` `이벤트 연결`

```
<p><span>안녕</span>
  <span class="bold">하세</span>
  <span>요</span></p>
```

상태를 바꾸는 주체가 둘로 나뉘어 **오류 발생 지점의 추적 난이도 증가**

B 대안 탐색

반복되는 예외 처리와 DOM 동기화 부담을 줄이면서, 기존 로직은 유지할 수 있는 기반 — DECISION

- 기준 1 **기본 편집 동작은 라이브러리가 처리** — 줄바꿈 · 삭제 · 붙여넣기 · 되돌리기
- 기준 2 **직접 만든 편집 로직은 유지** — 스타일 · 이미지 로직을 확장으로 이어 붙일 수 있는 구조

→

SELECTED

Tiptap

ProseMirror 기반 문서 모델 · 모든 변경은 트랜잭션으로 반영 · Mark · Node · Plugin 단위 확장 지원

→

Tiptap이 담당 — 기본 편집 기능
`StarterKit` `TextAlign` `TextStyle` `Color`

직접 구현 — 블로그 특화 기능
`CustomImage` `LinkPreview` `FontSize` `FontFamilyClass` `LinkPastePreview` ..

DECISION 전환 기준

기본 편집은 Tiptap에 맡기고, 직접 구현한 편집 로직은 확장(Extension)으로 이식

- 위임 — 입력 동작별 예외 처리와 DOM 동기화를 **문서 모델 · 트랜잭션 구조**에 위임
- 유지 — 텍스트 스타일 · 이미지 리사이즈 등 **직접 구현한 로직은 확장 단위로 재사용**

분석하고 구현한 에디터 기능, Tiptap 확장으로 다시 살리기

텍스트 노드 분할 · 비율 유지 이미지 리사이징 등 자체 에디터의 로직을 Mark · Node · Plugin 단위로 재구성했습니다.
Tiptap이 기본 제공하지 않는 기능은 확장 7가지로 구현해 채우고, 이미지 · 링크 · 서식 등 블로그에 필요한 편집 경험을 완성했습니다.

A 로직 이식

자체 에디터에서 구현한 편집 로직은 그대로 가져오고, Tiptap의 확장 구조에 맞춰 재구성 — PORTING

자체 에디터
텍스트 노드 분할
선택 구간을 잘라 **span을 직접 만들어 교체**

Mark
서식 단위로

TIPTAP 확장
FontSize · FontFamilyClass
선택 구간에 **서식 속성만 지정**하면,
글자 분할은 Tiptap이 알아서 처리

자체 에디터
비율 유지 리사이즈
data-id로 블록 식별 · **핸들 드래그로 크기 변경**

Node + NodeView
블록 단위로

TIPTAP 확장
CustomImage
같은 비율 계산을 그대로 사용하고,
바뀐 크기는 **에디터 문서 데이터에 저장**

TS lib/tiptap/CustomImage.ts · addNodeView

```ts
// 자체 에디터와 같은 비율 계산
aspect = startWidth / startHeight;

const onMouseMove = (e) => {
  const newWidth  = Math.max(32, startWidth + diffX);
  const newHeight = Math.max(32, newWidth / aspect);
  img.style.width = `${newWidth}px`;  // 드래그 중엔 화면만
};

const onMouseUp = () => {
  // 드래그가 끝나면 결과를 문서 모델에 반영
  tr.setNodeMarkup(pos, undefined, { ...attrs, width, height });
};
```

B Tiptap에 없던 기능, 확장 7가지로 구현

기본 제공 범위 밖의 요구사항을 개입 시점에 맞춰 Node · Mark · Plugin으로 설계 · 구현 — CUSTOM EXTENSIONS

NODE + NODEVIEW 2

- **CustomImage** — NodeView로 리사이즈 핸들 · 대표 지정 · 순서 이동 UI 구성
- **LinkPreview** — 링크 메타데이터를 속성으로 저장하는 카드 노드 설계

CUSTOM MARK 2

- **FontSize** — 글자 크기를 Mark 속성으로 저장하는 명령 추가
- **FontFamilyClass** — 폰트 교체 시 이전 폰트 클래스를 정리하는 로직

EXTENSION + PLUGIN 3

- **LinkPastePreview** — 붙여넣기를 가로채 URL 감지 → 메타데이터 요청 → 카드 삽입
- **MaxLength** — 반영 전 트랜잭션을 검사해 3,000자 초과 입력 차단
- **BlockDropGuard** — 드롭 위치를 계산해 이미지 · 링크 카드 겹침 방지

RESULT 개선 결과

기본 편집은 Tiptap, 블로그 특화 기능 7가지는 자체 설계한 확장으로 분리

- **재사용** — 자체 에디터에서 구현한 **스타일 · 이미지 로직을 확장 단위로 이식** — 분석하고 구현한 경험을 그대로 활용
- **구조** — 모든 편집이 **트랜잭션을 거쳐 문서 모델에 반영** — React 상태와 DOM의 동기화 부담 해소
- **유지보수** — 기본 편집 동작은 라이브러리가 담당하고 **관리할 코드는 확장 7개로 한정** — 자체 에디터를 계속 키우는 방식 대비 유지보수 범위 축소

초성 검색이 안 되는 공공 API, 학과 9,329개를 정적 데이터로 다루기

학과 검색에 **초성 검색**을 넣으려 전체 목록을 받아 저장해 두었지만 **첫 검색이 느리고** 목록이 **임의로 바뀔 위험**이 남았습니다. 그래서 목록을 **배포할 때 한 달에 한 번**만 받아 정적 데이터로 담았습니다.

사용한 공공 API **커리어넷 Open API** 학과 정보 · 9,329개



그 밖의 개선 수치

198 → **78ms** **입력 반응 시간 감소** 모바일
지연 렌더링(Lazy Rendering)으로 50개씩만 렌더링해 UI 프리징 감소

289 → **51KB** **내려받는 크기** gzip
빌드 시 gzip으로 압축해 전송량 약 82% 감소

개선 전 검색할 때마다 서버에 요청

1,537ms ← 학과를 입력하고 결과가 뜨기까지 (모바일 환경)

공공 API(커리어넷)는 **초성 검색을 지원하지 않아**,
이를 구현하려면 **학과 9,329개를 모두 받아와야** 했습니다.

검색할 때 일어나는 일



1차 시도 **공공 API 검색 기능 그대로 쓰기** 초성 검색 불가
커리어넷 Open API에 `searchTitle=ㅋㅍㅌ`처럼 초성을 보냈지만,
완성된 글자만 검색돼 초성으로는 학과를 찾을 수 없었습니다.

2차 시도 **전체 목록을 받아 잠시 저장해 두기** 첫 검색 1,537ms
전체 목록을 `TanStack Query`로 캐싱했지만,
첫 검색은 서버를 기다려야 했고, 새로고침하면 사라졌습니다.

3차 시도 **브라우저에 계속 저장해 두기** 사용자 목록 임의 수정 가능
`localStorage`(브라우저 저장 공간)로 새로고침해도 유지했지만,
첫 조회는 느렸고, 사용자가 목록을 직접 고칠 수 있었습니다.

개선 후 빌드 때 만든 정적 데이터에서 검색

55ms ← 같은 구간, 같은 기기로 측정 (모바일 환경) 약 96% 단축

개선 후에는 **빌드 단계에서 공공 API를 조회해 정적 데이터로 생성·압축**하므로,
네트워크 요청 없이 브라우저에서 바로 학과를 찾아 보여줍니다.

검색할 때 일어나는 일



최종 해결 **1,537ms → 55ms**

배포할 때 목록을 미리 담아두기

배포 직전 빌드 때 공공 API를 조회한 후, **정적 데이터로 생성·압축해 보관**합니다.
학과는 자주 바뀌지 않아 **한 달에 한 번만 갱신**합니다.
목록이 배포 파일에 포함돼 **사용자가 임의로 바꿀 수 없습니다.**

배포 빌드 · 학과 목록 갱신 스크립트

```
$ npm run build
› 운영 사이트 서비스주소/majors.meta.json 조회
  │ generatedAt: "2026-09-02"
  ├ 한 달 안 됨 → 갱신 건너뜀
  └ 한 달 지남 → 갱신 진행
› 공공 API 조회 → 학과명 정리 → 파일 저장
› gzip 압축  289KB → 약 51KB
› public/에 학과 파일 + majors.meta.json 함께 배포
✓ 학과 9,329개 갱신 완료  총 약 12초
```

구현 방식 생성일을 별도 저장소 없이 **지난 배포 결과물(majors.meta.json)에 기록**하고, 다음 빌드가 이를 읽어 갱신 여부를 판단합니다.



SIDE PROJECT · 2025.12 – 2026.05

YAMOYO

팀 초기 세팅 서비스

SWYP 웹 12기 최종 1위(대상)

FE 2 · BE 3 · DE 3 · PM 1

커밋 381 / 전체 591

머지 PR 39 / 전체 71

IN THIS PART

OVERVIEW
프로젝트 소개 · 맡은 영역 · 기술 구성 P.11

REAL-TIME
WebSocket 실시간 동기화와 연결 유지 P.12–13

AUTH ROUTING
이벤트 기반 인증 라우팅과 예외 처리 P.14–15

프로젝트 소개

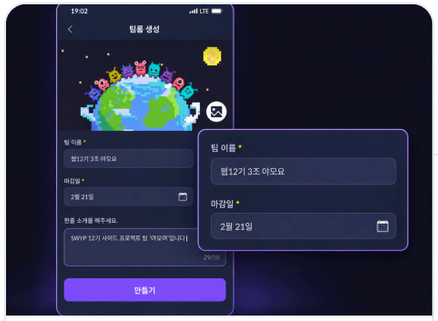
팀 프로젝트 시작에 필요한 결정을, 재밌고 빠르게 끝내는 팀 초기 세팅 서비스

fetch 기반의 API 요청 · 인증 계층, WebSocket 양방향 통신을 구성하고, 웹 푸시 알림과 코드 품질 환경을 함께 구축했습니다.

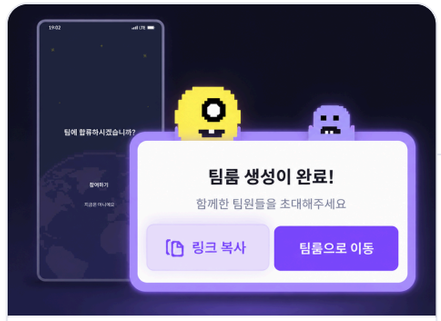
야모요 · yamoyo.kr | 팀 초기 세팅 서비스 | 2025.12 ~ 2026.05 | FE 2 · BE 3 · DE 3 · PM 1

A 서비스 흐름 팀이 처음 모여서 정해야 할 것들을 순서대로

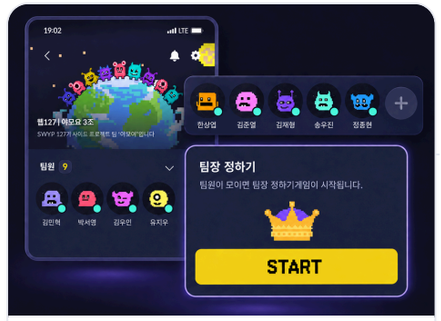
SERVICE



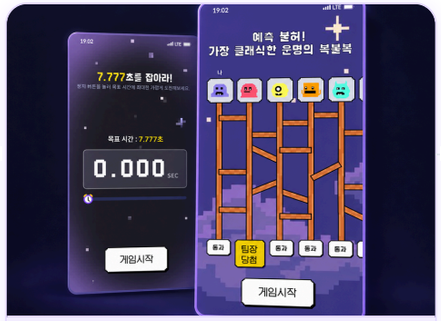
1
팀룸 개설
팀 이름 · 마감일 · 배너 설정



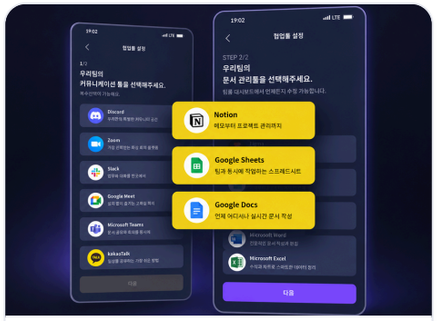
2
팀원 초대
초대 링크 · QR 코드로 합류



3
팀원 집결 · 시작
전원이 모이면 방장이 선정을 시작



4
팀장 정하기
지원 또는 사다리타기 · 타이밍 게임으로 선정



5
팀 규칙 · 협업툴
규칙에 합의하고 사용할 도구를 결정

맡은 영역

MY PART

1 **API 통신 계층 설계**
HTTP 라이브러리에 의존하지 않고 **fetch 기반 API 클라이언트**를 구축해 인증 · 에러 · 응답 파싱을 단일 진입점에서 처리

2 **인증 흐름 설계**
토큰 재발급과 인증 이벤트 라우팅, 로그인 후 진입 경로 복귀 · `shared/api/auth`

3 **실시간 팀장 선정**
STOMP WebSocket 연결 관리와 실시간 상태 동기화, 사다리 · 타이밍 게임 · `features/leader-game`

4 **웹 푸시 알림**
FCM 메시지를 **서버 상태 변경 신호로 활용** — 협업툴 승인 알림 수신 시 TanStack Query 캐시를 재조회해 팀룸 화면에 즉시 반영

5 **코드 컨벤션 도입**
ESLint · Prettier · Husky · lint-staged를 팀 컨벤션으로 정립하고, 커밋 단계에서 자동으로 검사되도록 구성

기술 구성

STACK

CORE	React 19 · TypeScript · Vite · Tailwind CSS
STATE	Zustand · TanStack Query
NETWORK	fetch 기반 API 클라이언트 · WebSocket · STOMP + SockJS · Firebase FCM
QUALITY	FSD 아키텍처 · ESLint · Prettier · Husky · lint-staged

PROJECT

대상 SWYP 웹 12기 최종 1위
기획 · 디자인 · 개발 · 배포 완주 프로그램

6개월 2025.12 ~ 2026.05
기획부터 배포 · 운영까지

381 개인 커밋 (전체 591)
프론트엔드 저장소 기준

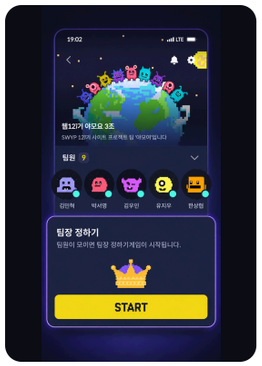
39 머지된 PR (전체 71)
코드 리뷰를 거쳐 병합

팀장 선정 과정을 모든 팀원 화면에 잇는 WebSocket 실시간 동기화

한 사람의 선택이 같은 방의 모든 팀원 화면에 즉시 반영되어야 했습니다.
연결과 구독을 공통 부모 한 곳으로 모으고, 화면은 서버가 내려주는 단계에 맞춰 그리도록 책임을 정리했습니다.

A 사용자에게 보이는 진행 과정

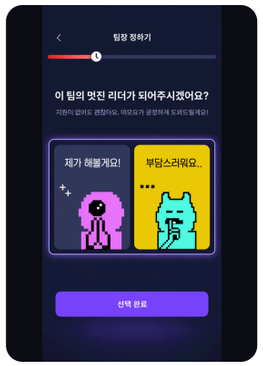
대기방에서 선정이 시작되면 팀장 선정 화면으로 이동합니다 — USER FLOW



1 게임 대기방

팀원의 접속 상태를 확인하고
선정 시작을 기다립니다

`USER_STATUS_CHANGE`



2 팀장 지원

지원 또는 넘기기를 선택해
서버로 전송합니다

`VOTE_UPDATED`

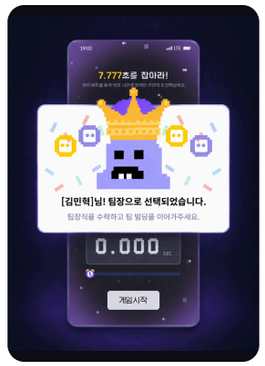


3 지원자가 없거나 여럿일 때

게임 진행

방장이 고른 미니게임으로
팀장을 선정합니다

`PHASE_CHANGE`



4 결과 안내

서버가 정한 결과를 모두에게
동일하게 안내

`GAME_RESULT`

1 구독은 한 곳에서, 받은 메시지는 이벤트 버스로 분배

BEFORE 대기방 · 지원 현황 · 게임 화면이 **각자 구독** → **AFTER** 공통 부모에서 **1회 구독** 후 이벤트 버스로 등록된 화면에만 분배

발행 · PUBLISH

- **방에 입장** `/pub/room/{id}/join`
- **팀장 지원 · 미지원** `/pub/room/{id}/volunteer`
- **게임 선택 (방장)** `/pub/room/{id}/select-game`

→ **야모요 서버**
요청을 처리하고
진행 상태를 전파

구독 · 공통 부모에서 1회

TeamRoomWsLayout
`/sub/room/{id}`

↓ `emitTeamRoomWsMessage()`

이벤트 버스 · ws-bus
`handlers: Set<(msg) => void>`
등록한 핸들러에만 메시지를 전달합니다

대기방 | 지원 현황 | 게임 화면

2 서버 메시지에 따른 화면 갱신

프론트엔드는 화면 이동과 렌더링을 담당합니다.

메시지 타입	화면 반영
`USER_STATUS_CHANGE`	팀원 **접속 · 이탈 표시** 갱신
`PHASE_CHANGE`	진행 **단계와 남은 시간** 갱신
`VOTE_UPDATED`	지원 · 미지원 **인원 현황** 갱신
`GAME_RESULT`	선정된 **팀장과 게임 데이터** 표시

ⓘ 선정 결과는 **서버가 판단**하고, 프론트엔드는 사용자 입력 전송과 화면 반영을 담당합니다.

POINT

1 연결 완료 전 호출을 잃지 않는 대기 큐
컴포넌트 마운트와 연결 완료 시점이 달라 구독 · 발행이 유실됐습니다.
연결 전 호출을 큐에 모아두었다가 연결 직후 순서대로 실행합니다

2 팀룸 범위를 벗어날 때만 연결 종료
팀룸 · 게임 화면 사이 이동이나 선정 진행 중에는 레이아웃이 연결을 유지하고,
팀룸 범위를 벗어나면 즉시 끊어 불필요한 연결을 남기지 않습니다

3 재접속해도 이어서 진행
연결이 끊겼다 돌아오면 재연결 직후 복구를 요청해,
현재 단계 · 남은 시간 · 지원 현황을 받아 진행 지점부터 이어갑니다

페이지 전환과 재렌더링으로 인한 WebSocket 연결 끊김 문제 해결

연결이 끊기는 순간 실시간 동기화도 멈춥니다.
연결 생명주기를 개별 페이지에서 공통 레이아웃으로 올려, 페이지 전환과 재렌더링에도 재연결 없이 통신하도록 구현했습니다.

PROBLEM

페이지 이동 · 재렌더링에 따른 재연결과 메시지 유실

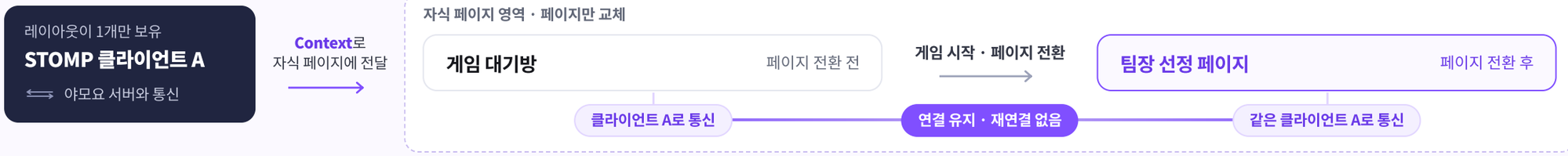
- 연결을 개별 페이지가 소유해, 페이지를 이동하면 **이동 전 페이지에 종속된 연결이 함께 끊김**
- 연결을 공통 부모로 옮긴 뒤에도 **재렌더링마다 클라이언트가 새로 생성**되어 재연결 · 재구독 반복

재연결 사이에 도착한 **진행 단계 · 지원 현황 메시지 유실**, 제한 시간이 있는 단계에서는 **재연결 지연만큼 선택 시간 손실**

SOLUTION

공통 레이아웃으로 연결 생명주기를 상위로 올리고, Context로 단일 STOMP 클라이언트 공유

공통 레이아웃 중첩 라우팅 — 두 페이지를 같은 레이아웃 아래에 배치해, **페이지를 옮겨도 레이아웃과 연결은 그대로 유지**



문제 재렌더링마다 클라이언트 재생성

새 클라이언트로 바뀌면 연결 로직이 다시 실행

기존 STOMP 클라이언트 A → 새로 생성 STOMP 클라이언트 B

기존 연결 종료 · 새 클라이언트로 재연결

해결 STOMP 클라이언트 재사용

useMemo로 accessToken이 바뀔 때만 새로 생성하고, 그 외 **재렌더링에는 기존 클라이언트 재사용**

기존 STOMP 클라이언트 A → 재사용 STOMP 클라이언트 A

연결 유지 · 불필요한 종료 · 재연결 방지

RESULT

페이지 전환당 재연결	재연결 지연	전환 1회당 연결 · 구독 · 복구 요청	재연결 중 메시지 유실	화면별 구독 → 레이아웃 1회 구독
~~1회~~ → **0회**	~~약 0.7~1.2초~~ → **0초**	~~약 36건~~ → **0건**	~~발생~~ → **0건**	~~3곳~~ → **1곳**
		6인 방 기준	메시지 유실 없는 단계 동기화	구독 로직 재사용 · 유지보수성 향상

CODE features/leader-game/ws/layout/TeamRoomWsLayout.tsx

SPA 전환과 상태 전달의 이점을 지키는 신호 발행 · 구독 기반 인증 라우팅

인증 만료와 온보딩 미완료 상태를 API 계층이 '신호'로만 발행하고,
React 내부의 인증 보호 레이어가 그 신호를 구독해 화면 이동을 담당하도록 다시 설계했습니다.

야모요 · 팀 초기 세팅 서비스 | React 19 · TypeScript | fetch 기반 API 클라이언트 | Zustand · TanStack Query | React Router

A 문제 배경 요청 계층이 화면 이동까지 떠안으면서 생긴 한계 — BEFORE

API 요청 / 응답 해석
React 컴포넌트 외부(.ts)에서 동작

→

인증 예외 감지
토큰 만료 · 약관 · 프로필 미완료

→

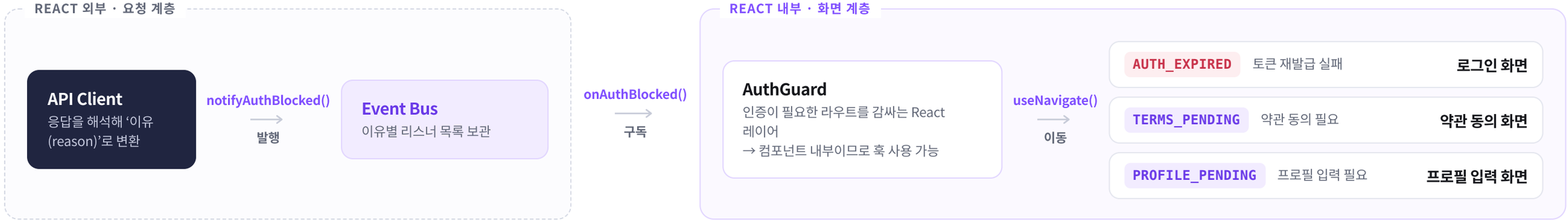
화면 이동이 필요
로그인 · 약관 · 프로필 화면으로

→

useNavigate 사용 불가
window.location → 전체 새로고침

window.location으로 이동하면 **전체 새로고침이 발생해 SPA 전환과 상태 전달의 이점을 잃고**, 만료 · 약관 · 프로필 분기를 화면마다 처리하면 인증 흐름이 페이지 수만큼 흩어집니다.

B 해결 흐름 상태 감지는 요청 계층, 화면 이동은 화면 계층 — AFTER



API 계층은 라우팅하지 않고 **'무슨 상태인지'만 발행**하도록 책임을 줄이고, 보호 라우트를 감싸는 AuthGuard가 그 신호를 구독해 **화면 이동을 전담**합니다.

RESULT

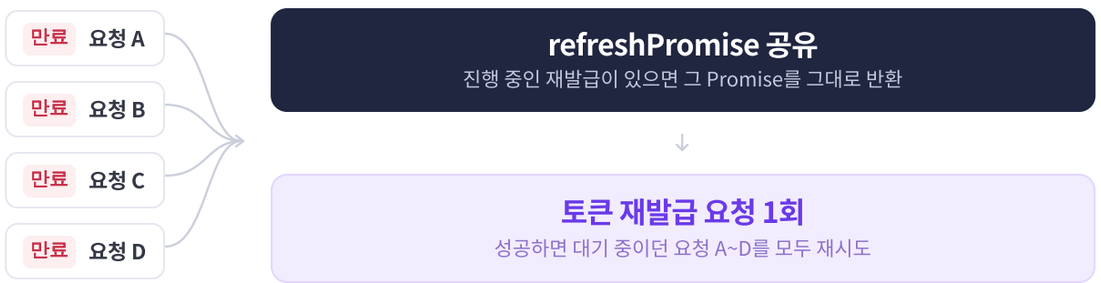
- **인증 분기를 한 곳으로** — 흩어져 있던 처리를 AuthGuard로 모아 중복 제거
- **전체 새로고침 없이 이동** — SPA 라우팅으로 경로와 상태를 그대로 전달
- **불필요한 리렌더링 없음** — 전역 구독과 달리 해당 이유를 구독한 곳만 실행

동시에 쏟아지는 예외, 어떻게 한 번만 처리할까?

한 화면이 여러 API를 동시에 호출하는 실제 사용 환경에서는 토큰 재발급과 화면 이동이 중복으로 실행됩니다. Promise 병합과 우선순위 규칙으로 정리했습니다.

1 동시에 발생한 토큰 만료 예외를 재발급 1회로 병합

진행 중인 재발급 Promise를 공유해, 요청이 몇 개든 재발급 API는 한 번만 호출되도록 했습니다.



2 여러 이유가 겹쳐도 우선순위 하나만 처리

이유마다 우선순위와 처리 플래그를 두어, 중복 화면 이동과 화면 흔들림이 생기지 않도록 했습니다.



3 우선순위와 플래그를 적용한 이벤트 발행

이해를 돕기 위해 실제 코드를 축약하였습니다.

TS src/shared/api/auth/event-bus.ts

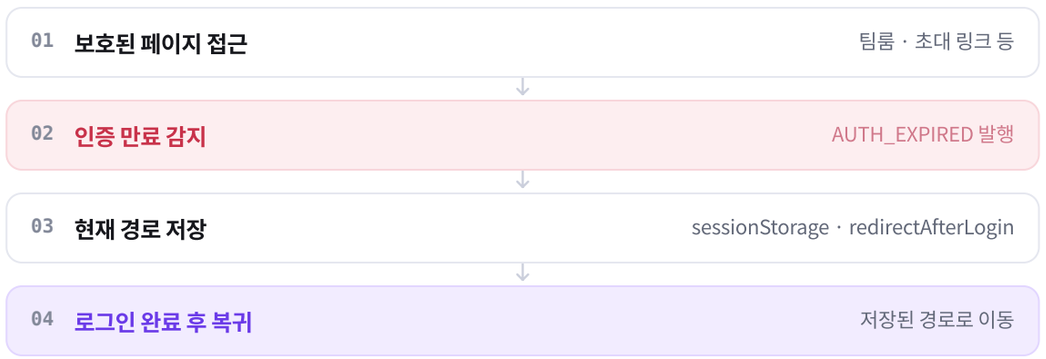
```ts
// 이유별 우선순위 — 값이 작을수록 우선순위가 높다
const priority = { AUTH_EXPIRED: 0, TERMS_PENDING: 1, PROFILE_PENDING: 2 };

export function notifyAuthBlocked(reason: AuthBlockReason) {
  // 처리 중인 이유가 있다면, 더 높은 우선순위일 때만 교체
  if (activeReason && priority[reason] >= priority[activeReason]) return;
  activeReason = reason;

  if (notified[reason]) return;   // 같은 이유는 한 번만 처리
  notified[reason] = true;
  listeners[reason].forEach((listener) => listener());
}
```

4 로그인 후, 보던 페이지로 복귀

인증이 끊긴 지점을 기억해 로그인 뒤 흐름이 끊기지 않도록 했습니다.



! 초대 링크처럼 먼저 저장된 경로는 **덮어쓰지 않도록** 하고, 온보딩 경로는 저장 대상에서 제외했습니다.

RESULT

- **동시에 쏟아진 토큰 만료 예외를 재발급 한 번으로 병합**
  중복 재발급 요청과 토큰 불일치를 방지했습니다
- **흩어져 있던 인증 예외를 한 곳에서 관리**
  로그인 만료 · 약관 동의 · 프로필 입력을 같은 규칙으로 처리
- **지금 필요한 화면으로만 한 번 전환**
  예외가 겹쳐도 화면 흔들림 없이 일관된 인증 흐름 유지

THANK YOU

읽어주셔서 감사합니다

포트폴리오에 담지 못한 구현 과정과 시행착오는
블로그에 자세히 기록해 두었습니다.

MORE DETAILS

섹션별 구현 과정 기록

01 인증 콜백 처리 /11 /12

02 에디터 확장 구조 /13 /14

04 YAMOYO 인증 흐름 /17

EMAIL
jaqwe2301@gmail.com

GITHUB
github.com/jaqwe2301

BLOG
mini-frontend.tistory.com